Inheritance vs Composition vs Aggregation

This is an **important OOP design question** because interviewers often ask:

> **"When should you use inheritance vs composition?"**

The core idea is:

- **Inheritance → IS-A**
- **Composition → HAS-A**
- **Aggregation → HAS-A, but weaker ownership**

---

1. Inheritance — "IS-A"

Inheritance represents an **is-a relationship**.

```
Dog IS-A Animal
Car IS-A Vehicle
Manager IS-A Employee
```

Example:

```
class Animal {
public:
    void eat() {
        cout << "Eating";
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Barking";
    }
};
```

Here:

```
Dog → Animal
```

A `Dog` **is an** `Animal`.

So inheritance makes sense.

```
Dog d;

d.eat();   // inherited
d.bark();  // own behavior
```

Interview definition

> Inheritance models an **IS-A relationship**, where a derived class specializes a base class.

---

2. Composition — "HAS-A" with strong ownership

Composition means one class **contains another object as a part of itself**.

```
Car HAS-A Engine
House HAS-A Room
Computer HAS-A CPU
```

Example:

```cpp
class Engine {
public:
    void start() {
        cout << "Engine started";
    }
};

class Car {
private:
    Engine engine;  // Composition

public:
    void startCar() {
        engine.start();
    }
};
```

Here:

```
Car
 └── Engine
```

`Car` has an `Engine`.

Why is composition "strong"?

The contained object is generally **owned by the containing object**.

```cpp
class Car {
    Engine engine;
};
```

When a `Car` is destroyed, its `Engine` is automatically destroyed too.

```
Car created
   ↓
Engine created

Car destroyed
   ↓
Engine destroyed
```

The `Engine`'s lifetime is tied to the `Car`.

Interview definition

> Composition is a strong HAS-A relationship where the containing object owns the contained object and controls its lifetime.

---

3. Aggregation — "HAS-A" with weak ownership

Aggregation is also a **HAS-A relationship**, but the contained object can exist independently.

Example:

```
Department HAS-A Professor
```

But a professor can exist without that particular department.

```cpp
class Professor {
};

class Department {
private:
    Professor* professor;

public:
    Department(Professor* p) : professor(p) {}
};
```

Now:

```
Professor p;

{
    Department d(&p);
}
```

When `d` is destroyed:

```
Department destroyed
Professor still exists
```

The `Department` **doesn't own** the `Professor`.

Interview definition

> Aggregation is a weak HAS-A relationship where the contained object can exist independently of the container.

---

4. Composition vs Aggregation

This is the distinction you should really understand.

Composition

```
Car
 └── Engine
```

Car **owns** Engine.

```
class Car {
    Engine engine;
};
```

Destroy `Car` → `Engine` is destroyed.

Aggregation

```
Department ──→ Professor
```

Department **uses/references** Professor but doesn't own it.

```
class Department {
    Professor* professor;
};
```

Destroy `Department` → Professor can continue existing.

---

5. The easiest mental model

Think about **lifetime**.

RelationshipMeaningOwnershipLifetimeInheritanceIS-AN/ABase/derived relationshipCompositionHAS-AStrongPart depends on wholeAggregationHAS-AWeak/no ownershipPart can exist independently

6. Real-world example

Imagine a university system.

Inheritance

```
Person
 /   \
Student Professor
```

A:

```
Student IS-A Person
Professor IS-A Person
```

So inheritance makes sense.

Composition

```
Car
 └── Engine
```

A car owns its engine.

So:

```
class Car {
    Engine engine;
};
```

Aggregation

```
University
 ————————→ Professor
```

A professor can exist independently of a particular university object.

So the university can reference a professor without owning its lifetime.

---

7. Why prefer composition over inheritance?

This is a **very common interview question**.

Suppose:

```
class Engine {
public:
    void start();
};

class Car {
private:
    Engine engine;
};
```

You can change the implementation of `Engine` without making `Car` inherit from it.

Composition provides **looser coupling**.

Inheritance creates a stronger relationship:

```
Car IS-A Engine ❌
```

which doesn't make semantic sense.

The rule is:

> **Don't use inheritance just to reuse code. Use it when there is a genuine IS-A relationship.**

For code reuse, composition is often preferable.

---

8. Composition vs Inheritance

Suppose you have:

```
class Engine {
public:
    void start() {}
};
```

❌ Bad inheritance

```
class Car : public Engine {
};
```

This says:

```
Car IS-A Engine
```

which is false.

☑ Good composition

```
class Car {
private:
    Engine engine;

public:
    void start() {
        engine.start();
    }
};
```

This says:

```
Car HAS-A Engine
```

which is correct.

---

9. Composition is often more flexible

Imagine:

```
class Engine {
public:
    virtual void start() = 0;
};
```

Different engines:

```
class PetrolEngine : public Engine {};
class ElectricEngine : public Engine {};
```

Now:

```
class Car {
private:
    unique_ptr<Engine> engine;

public:
    Car(unique_ptr<Engine> e)
        : engine(std::move(e)) {}
};
```

You can create:

```
Car petrolCar(make_unique<PetrolEngine>());
Car electricCar(make_unique<ElectricEngine>());
```

The `Car` doesn't need to inherit from either engine.

This is a powerful example of **composition + polymorphism**.

10. Interview trap: Composition vs Aggregation in C++

There isn't a special C++ keyword for either.

You don't write:

```
composition
```

or:

```
aggregation
```

They're **design concepts**, not language features.

For example:

```
class Car {
    Engine engine;
};
```

strongly expresses composition.

Whereas:

```
class Department {
    Professor* professor;
};
```

can express aggregation, depending on the ownership/design semantics.

🎯 Interview Questions

Q1. What is the difference between IS-A and HAS-A?

IS-A → inheritance

```
Dog IS-A Animal
```

HAS-A → composition/aggregation

```
Car HAS-A Engine
```

Q2. Composition vs aggregation?

> Both represent HAS-A relationships. Composition implies strong ownership and dependent lifetime, while aggregation represents weaker association where the contained object can exist independently.

Q3. Why prefer composition over inheritance?

> Composition generally provides lower coupling, better flexibility, and allows behavior to be changed by replacing contained objects rather than modifying the inheritance hierarchy.

Q4. Is composition inheritance?

No.

Composition:

```
class Car {
    Engine engine;
};
```

Inheritance:

```
class Dog : public Animal {};
```

Q5. Can composition use inheritance?

Yes.

For example:

```
class Engine {
public:
    virtual void start() = 0;
};

class PetrolEngine : public Engine {};
class ElectricEngine : public Engine {};

class Car {
    unique_ptr<Engine> engine;
};
```

Here you have **composition + polymorphism + inheritance** together.

🔥 Final cheat sheet

```
INHERITANCE
    ↓
"IS-A"
    ↓
Dog IS-A Animal
    ↓
Strong type relationship


COMPOSITION
    ↓
"HAS-A"
    ↓
Car HAS-A Engine
    ↓
Strong ownership
    ↓
Part's lifetime tied to whole


AGGREGATION
    ↓
"HAS-A"
    ↓
Department HAS-A Professor
    ↓
Weak ownership
    ↓
Part can exist independently
```

The interview rule to memorize

> **Use inheritance for IS-A relationships and composition/aggregation for HAS-A relationships. Prefer composition when you need flexible reuse without creating a rigid inheritance hierarchy.**
